Robotics 2: The Ultimate Midterm Guide

Theory & Step-by-Step Exercise Solver

Based on Prof. De Luca's Lectures

April 10, 2026

Contents

Part 1: Theory Core Concepts

1 Kinematic Calibration

Calibration improves robot accuracy by updating nominal Denavit-Hartenberg (DH) parameters using external measurements. The actual end-effector position p depends on the parameters $\xi = [\alpha, a, d, \theta]^T$. Linearizing the direct kinematics around the nominal parameters yields the basic calibration equation:

$$\Delta p = \Phi(q)\Delta\xi \tag{1}$$

where $\Delta p = p_{measured} - p_{nominal}$ is the Cartesian error, $\Delta\xi$ is the vector of parameter errors, and $\Phi(q)$ is the **Regressor Matrix**.

Building the Regressor Matrix

The matrix Φ is built by taking the partial derivatives of the direct kinematics $k(q)$ with respect to the specific uncertain parameters, evaluated at their nominal values:

$$\Phi_i(q) = \left.\frac{\partial k(q)}{\partial \xi_i}\right|_{nom} \tag{2}$$

For multiple measurements at different configurations $q_1, q_2, \ldots, q_N$, we stack the equations and solve using the pseudoinverse: $\Delta\xi = \Phi^{\#}_{stacked}\Delta p_{stacked}$.

2 Kinematic Redundancy

A robot is redundant if the dimension of the joint space n is strictly greater than the dimension of the task space m ($n > m$). The differential kinematics is $J(q)\dot{q} = \dot{x}$. Since J is $m \times n$, there are infinite solutions.

Redundancy Resolution Methods

- **Pseudoinverse (Minimum Norm):** $\dot{q}_{PS} = J^{\#}\dot{x} = J^T(JJ^T)^{-1}\dot{x}$. Minimizes $\|\dot{q}\|^2$.
- **Weighted Pseudoinverse:** $\dot{q}_W = W^{-1}J^T(JW^{-1}J^T)^{-1}\dot{x}$. Minimizes $\dot{q}^T W\dot{q}$.
- **Null-Space Projection:** $\dot{q} = J^{\#}\dot{x} + (I - J^{\#}J)\dot{q}_0$. The term $(I - J^{\#}J)$ projects any desired velocity $\dot{q}_0$ into the null space of J (self-motion).
- **Inertia-Weighted Pseudoinverse (Minimizes Kinetic Energy):** If we choose the weight matrix $W = M(q)$ (the inertia matrix), the weighted pseudoinverse $\dot{q}_M = M^{-1}J^T(JM^{-1}J^T)^{-1}\dot{x}$ minimizes the instantaneous kinetic energy $T = \frac{1}{2}\dot{q}^T M(q)\dot{q}$.

3 Lagrangian Dynamics

The dynamic model of a rigid manipulator is derived from the Lagrangian $\mathcal{L}(q, \dot{q}) = T(q, \dot{q}) - U(q)$, where T is kinetic energy and U is potential energy. The Euler-Lagrange equations yield:

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau \tag{3}$$

- $M(q)$: $n \times n$ symmetric, positive definite Inertia Matrix.
- $c(q, \dot{q})$: Coriolis and centrifugal forces vector. $c(q, \dot{q}) = S(q, \dot{q})\dot{q}$.

- $g(q)$: Gravity vector. $g(q) = (\frac{\partial U}{\partial q})^T$.

3.1 Actuator Dynamics and Friction

In a real manipulator, torques are provided by motors through reduction gears. If n_{ri} is the gear ratio of the i-th joint and I_{mi} is the motor rotor inertia, the kinetic energy of the motors adds a constant diagonal matrix to the inertia matrix:

$$M_{tot}(q) = M(q) + \text{diag}(n_{ri}^2 I_{mi}) \tag{4}$$

Additionally, the real torque τ must compensate for joint friction. The complete model becomes:

$$M_{tot}(q)\ddot{q} + c(q, \dot{q}) + g(q) + F_v\dot{q} + F_c\text{sgn}(\dot{q}) = \tau \tag{5}$$

where F_v is the viscous friction diagonal matrix and F_c is the Coulomb friction diagonal matrix.

3.2 Important Properties of the Dynamic Model

1. **Skew-Symmetry:** The matrix $\dot{M}(q) - 2S(q, \dot{q})$ is skew-symmetric IF S is derived using Christoffel symbols. This implies $x^T(\dot{M} - 2S)x = 0 \quad \forall x$.

2. **Linear Parameterization:** The dynamic model is linear with respect to a set of constant dynamic parameters (masses, lengths, inertias). It can be written as $Y(q, \dot{q}, \ddot{q})\mathbf{a} = \tau$, where Y is the dynamic regressor and $\mathbf{a}$ is the vector of dynamic coefficients.

Part 2: How to Solve Midterm Exercises

This section provides step-by-step algorithms to solve the exact typologies of exercises found in the exams.

4 Type A.1: Task Priority and Task Augmentation

Algorithm: Solving Task Priority (TP) Problems **Goal:** Execute a primary task $\dot{x}_1$ perfectly, and a secondary task $\dot{x}_2$ as best as possible using the remaining degrees of freedom.

1. Identify the Jacobians J_1 and J_2.
2. Calculate the pseudoinverse of the primary task: $J_1^{\#} = J_1^T(J_1 J_1^T)^{-1}$.
3. Calculate the Null-Space projector of the primary task: $P_1 = (I - J_1^{\#} J_1)$.
4. The joint velocity for the primary task is: $\dot{q}_1 = J_1^{\#}\dot{x}_1$.
5. Calculate the residual error for the second task: $e_2 = \dot{x}_2 - J_2\dot{q}_1$.
6. Project J_2 into the null space of J_1: $\tilde{J}_2 = J_2 P_1$.
7. The total solution is: $\dot{q}_{TP} = \dot{q}_1 + \tilde{J}_2^{\#} e_2$.

Exam Tip / Watch out!

If the augmented Jacobian $J_A = \begin{bmatrix} J_1 \\ J_2 \end{bmatrix}$ is FULL RANK, then the Task Priority solution exactly matches the Task Augmentation (TA) solution: $\dot{q}_{TA} = J_A^{\#} \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix}$. Priority only matters when tasks conflict (i.e., algorithmic singularity).

5 Type A.2: Projected and Reduced Gradient

Algorithm: Projected Gradient (PG) **Goal:** Execute a task $\dot{x}$ while maximizing/minimizing a scalar objective function $H(q)$ (e.g., distance from joint limits).

1. Compute the unconstrained solution: $\dot{q}_{task} = J^{\#}\dot{x}$.
2. Compute the gradient of your objective function: $\nabla_q H = \begin{bmatrix} \frac{\partial H}{\partial q_1} & \frac{\partial H}{\partial q_2} & \dots \end{bmatrix}^T$.
3. Define desired null-space velocity: $\dot{q}_0 = \pm\alpha\nabla_q H$ (use $+$ to maximize, $-$ to minimize, with $\alpha > 0$).
4. Calculate Null-Space projector: $P = (I - J^{\#}J)$.
5. Total velocity: $\dot{q}_{PG} = \dot{q}_{task} + P\dot{q}_0$.

Algorithm: Reduced Gradient (RG) **Goal:** Same as PG, but computationally lighter (no pseudoinverse).

1. Partition the Jacobian into a square non-singular matrix J_a $(m \times m)$ and J_b $(m \times (n-m))$: $J = [J_a \mid J_b]$.
2. Partition joints accordingly: q_a (dependent) and q_b (independent).

3. Compute the *Reduced Gradient*: $\nabla_{q_b} H' = [-(J_a^{-1} J_b)^T \mid I] \nabla_q H$.

4. Update independent joints: $\dot{q}_b = \pm \alpha \nabla_{q_b} H'$.

5. Solve for dependent joints to ensure task execution: $\dot{q}_a = J_a^{-1}(\dot{x} - J_b \dot{q}_b)$.

6 Type B: SNS Method (Hard Bounds)

Algorithm: Saturation in the Null Space (SNS) **Goal:** Find joint velocities that realize the task but strictly respect bounds $|\dot{q}_i| \leq V_{max}$.

1. **Initial guess:** Calculate unconstrained solution $\dot{q} = J^{\#} \dot{x}$.

2. **Check bounds:** Does any element of $\dot{q}$ exceed its maximum? If NO, done. If YES, proceed to 3.

3. **Saturate:** Take the joint k that most violates the bound. Set $\dot{q}_k = V_{max} \times \text{sign}(\dot{q}_k)$.

4. **Reduce system:** Remove column k from J to get J_{-k}. Modify the task: $\dot{x}_{new} = \dot{x} - J_k \dot{q}_k$.

5. **Re-solve:** Calculate new velocity for remaining joints: $\dot{q}_{-k} = J_{-k}^{\#} \dot{x}_{new}$.

6. **Re-check:** Check if the new combined $\dot{q}$ satisfies all bounds. If not, repeat.

7 Type C: Deriving the Dynamic Model

Algorithm: Deriving the Inertia Matrix M **Goal:** Find the symmetric matrix $M(q)$ from kinetic energy $T = \frac{1}{2} \dot{q}^T M(q) \dot{q}$.

1. For each link i, compute the linear velocity of its Center of Mass, v_{ci}, and its angular velocity, ω_i.

2. Write Kinetic Energy for each link: $T_i = \frac{1}{2} m_i \|v_{ci}\|^2 + \frac{1}{2} \omega_i^T I_{ci} \omega_i$.

3. Sum them up: $T_{tot} = \sum T_i$.

4. Expand T_{tot} and group terms into the quadratic form $\frac{1}{2} \sum \sum m_{ij}(q) \dot{q}_i \dot{q}_j$. The coefficients form $M(q)$.

Algorithm: Deriving Coriolis and Centrifugal terms c **Method: Christoffel Symbols (Fast Matrix form)** Let M_k be the k-th column of $M(q)$. The k-th element of vector c is $c_k = \dot{q}^T C_k(q) \dot{q}$, where:

$$C_k(q) = \frac{1}{2} \left[\frac{\partial M_k}{\partial q} + \left(\frac{\partial M_k}{\partial q} \right)^T - \frac{\partial M}{\partial q_k} \right] \tag{6}$$

Stack the resulting scalars to get the vector $c(q, \dot{q})$.

Algorithm: Deriving Gravity Vector g

1. Find the height y_{ci} (component aligned with gravity) of each CoM.

2. Write total potential energy: $U(q) = \sum m_i g_0 y_{ci}$.

3. Differentiate: $g(q) = \left(\frac{\partial U(q)}{\partial q} \right)^T$.

8 Type D: Skew-Symmetry and Factorizations

Algorithm: Finding Factorizations S1 and S2 **Goal:** Find matrices S such that $S\dot{q} = c$, one making $\dot{M} - 2S$ skew-symmetric (S_1), and one not (S_2).

1. **Skew-symmetric S_1:** Stack the Christoffel matrices you found earlier!

$$S_1(q, \dot{q}) = \begin{bmatrix} \dot{q}^T C_1(q) \\ \dot{q}^T C_2(q) \\ \cdots \end{bmatrix} \tag{7}$$

2. **NON skew-symmetric S_2:** Add a dummy matrix $S_0(\dot{q})$ to S_1 such that $S_0(\dot{q})\dot{q} = 0$. For a 2-DOF robot, choose:

$$S_0(\dot{q}) = \begin{bmatrix} 0 & -\dot{q}_2 \\ \dot{q}_2 & 0 \end{bmatrix}$$

Then $S_2 = S_1 + S_0$. Since $S_0\dot{q} = 0$, $S_2\dot{q} = c(q, \dot{q})$, but $\dot{M} - 2S_2$ will NOT be skew-symmetric.

9 Type E: Linear Parameterization and Identification

Algorithm: Extracting Regressor Y and parameters a **Goal:** Rewrite $M\ddot{q} + c + g = \tau$ as $Y(q, \dot{q}, \ddot{q})\mathbf{a} = \tau$.

1. Identify all blocks of constants (masses, lengths, inertias). Group terms that always appear together (e.g., $I_1 + m_1 l_{c1}^2$) into a single dynamic coefficient a_k.
2. Define your vector $\mathbf{a} = [a_1, a_2, \ldots, a_p]^T$.
3. Rewrite the equations of motion isolating the variables.
4. Extract the variables into the $n \times p$ matrix Y.

Algorithm: Dynamic Identification Once you have $Y\mathbf{a} = \tau$, to find the true parameters $\mathbf{a}$ experimentally:

1. Sample $q, \dot{q}, \ddot{q}$ and τ at N time instants along an exciting trajectory.
2. Build the stacked Regressor Matrix $\bar{Y}$ ($nN \times p$) and Torque vector $\bar{\tau}$ ($nN \times 1$).
3. Solve using Least Squares: $\hat{\mathbf{a}} = \bar{Y}^{\#}\bar{\tau} = (\bar{Y}^T\bar{Y})^{-1}\bar{Y}^T\bar{\tau}$.

10 Type F: Dynamic Redundancy Resolution

Algorithm: Minimizing Torque Norm **Goal:** Find the joint acceleration $\ddot{q}$ (or torque τ) that executes a Cartesian task $\ddot{x}$ while minimizing the norm of the joint torques $\|\tau\|^2$.

1. Write the task at acceleration level: $J\ddot{q} = \ddot{x} - \dot{J}\dot{q}$.
2. Substitute the dynamic model $\ddot{q} = M^{-1}(\tau - c - g)$ into the task equation.
3. The problem becomes finding τ to satisfy the task. The minimum norm solution is obtained using the inertia-weighted pseudoinverse:

$$\tau_{min} = (JM^{-1})^{\#}(\ddot{x} - \dot{J}\dot{q} + JM^{-1}(c + g)) \tag{8}$$

11 Type G: Time Scaling under Torque Bounds

Algorithm: Uniform Time Scaling **Goal:** Find scale factor $k \geq 1$ to satisfy torque bounds $|u_i| \leq U_{max}$ without altering the geometric path.

1. Determine the max required torque $\tau_{orig}(t)$ during the given trajectory using Inverse Dynamics.
2. Separate the torque into Gravity terms (g) which DO NOT scale, and Inertial/Coriolis terms (τ_{dyn}) which scale with $1/k^2$.
3. Set up the inequality for the most critical joint:
$$\left| \frac{\tau_{dyn}}{k^2} + g \right| \leq U_{max} \tag{9}$$
4. Solve for k:
$$k \geq \sqrt{\frac{|\tau_{dyn}|}{U_{max} - |g|}} \tag{10}$$
5. The new feasible execution time is $T_s = k \cdot T_{old}$.